

端侧 Infra 第三期

从 2 帧到 15 帧

端侧推理性能优化的完整链路

今天的议程

- ① 拆延迟：推理时间到底花在哪
- ② 多模态的瓶颈：为什么不能照搬 LLM 优化套路
- ③ 五刀优化：每一步收益与代价的权衡
- ④ 架构层解法：分层系统如何绕过延迟瓶颈
- ⑤ 上线之后的坑：热降频、内存峰值、异常兜底
- ⑥ 项目 + 面试：怎样讲清楚优化工作

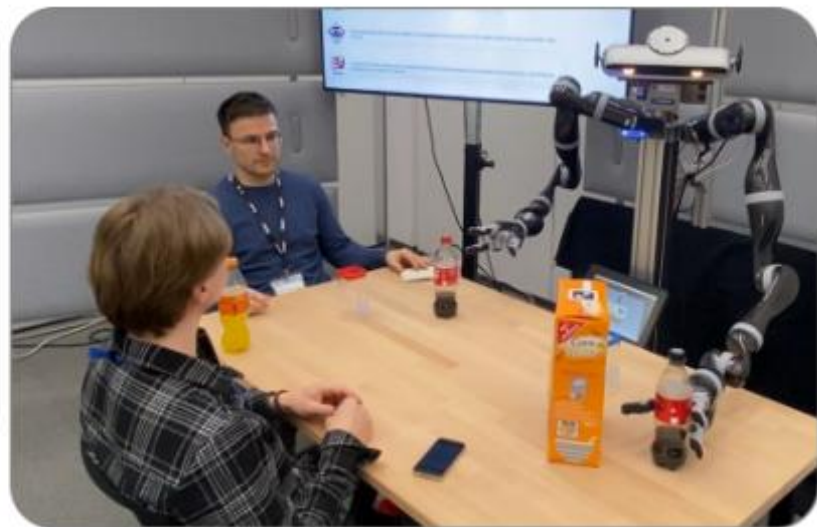
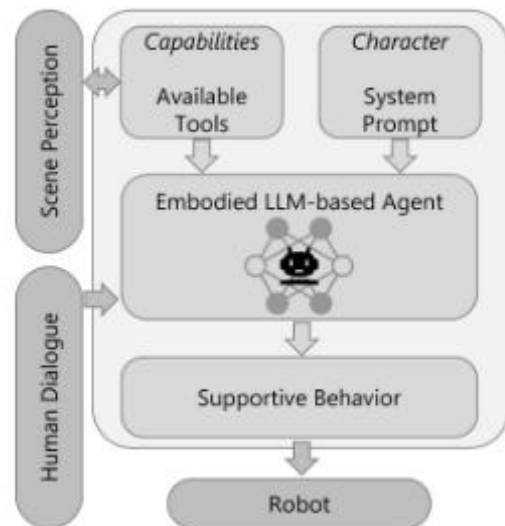
0

问题背景

机器人控制的魔数：

- 控制环频率： 50 Hz (20 ms 一个指令)
- VLM 在端侧的延迟： 2000-5000 ms
- 用户期望： <100 ms

25 倍的性能鸿沟



真实案例：机器人视觉问答

问题 1：首 Token 延迟太长

原始 800 ms → 目标 100 ms

问题 2：运行时性能不稳定

第 1 分钟快，第 10 分钟卡（热降频）

问题 3：单一异常导致级联失败

VLM 异常输出 → 机器人非法动作

1

拆延迟

所有优化的第一步：

把时间拆成 10 个部分，

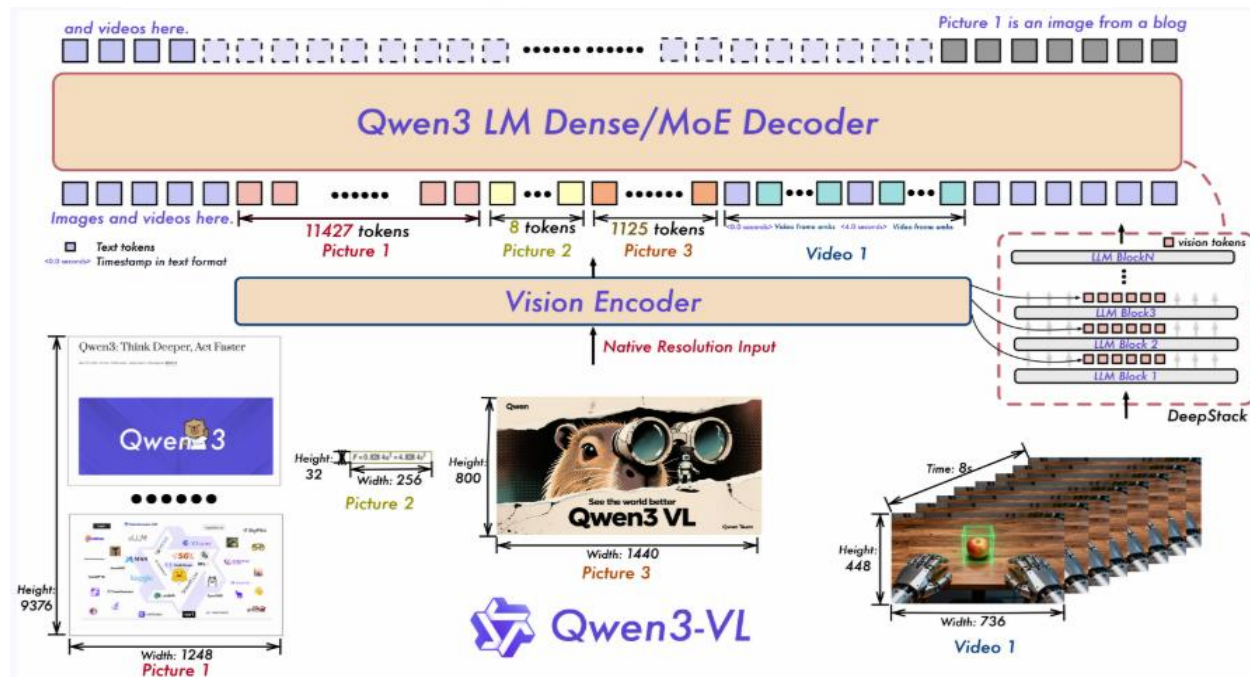
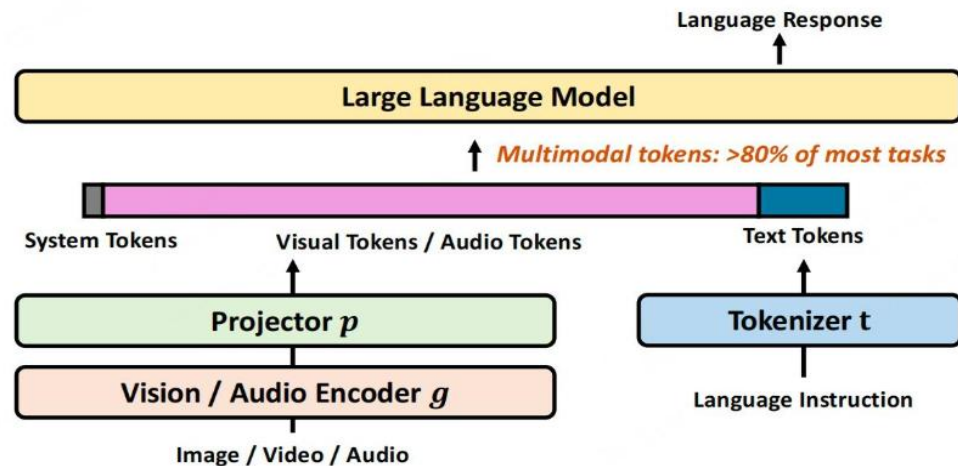
找清楚哪个部分最大。

一次 VLM 推理的延迟分布

假设：7B VLM, 1 张 1080p 图 + 一句话 → 50 token

- ① 图像采集 + 预处理：100-150 ms
- ② ViT 视觉编码：180-250 ms
- ③ Prefill (处理所有 token)：400-600 ms ← 大头
- ④ Decode (生成 50 个 token)：150-200 ms
- ⑤ 后处理 + 指令下发：50-80 ms

总计：~900-1200 ms



多模态 vs 纯文本 LLM

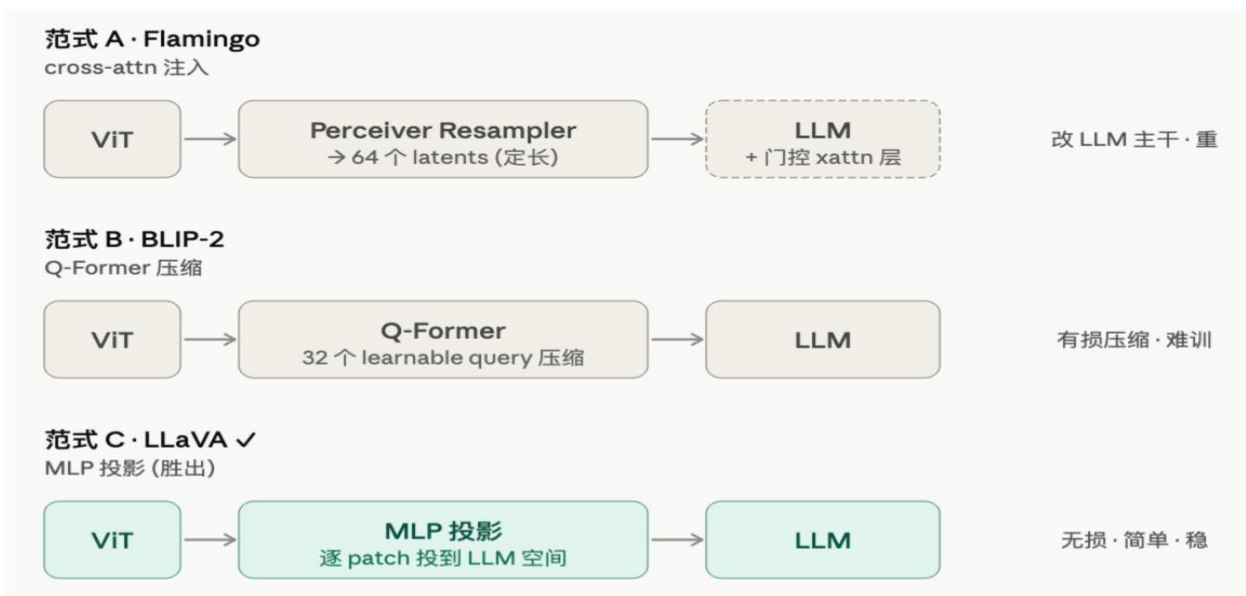
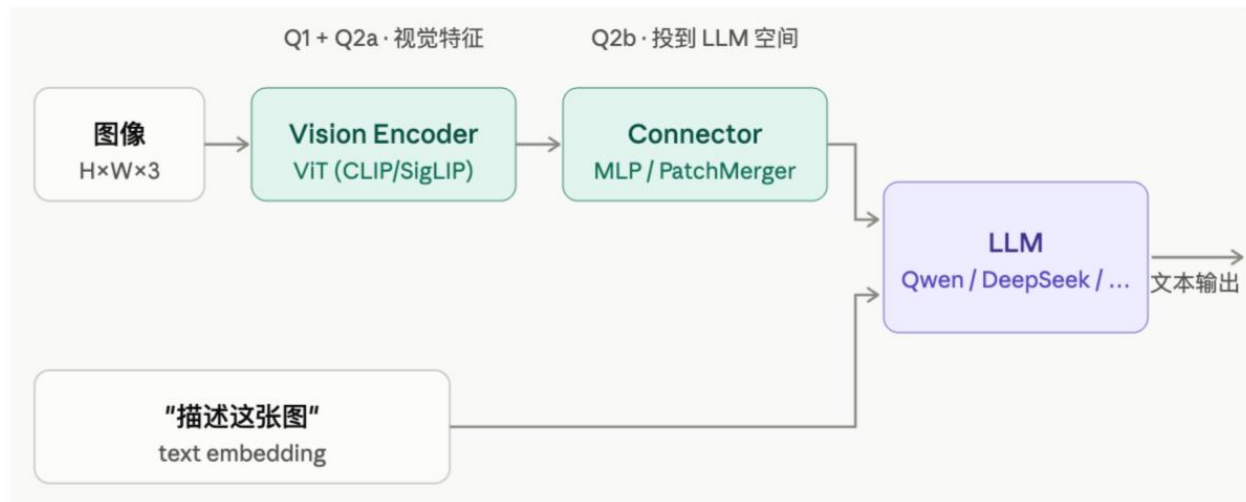
纯文本 LLM 的延迟分布:

- Prefill 快, Decode 慢
- 优化重心 → Decode (KV Cache、投机解码)

多模态 VLM 的延迟分布:

- 视觉编码必须做 (1000+ token)
- Prefill 必须做 (视觉 token 多)
- Decode 只占 15-20%
- 优化重心 → 视觉编码 + Prefill

常见错误: 照搬 LLM 的"只优化 Decode"套路 $\times$



隐形杀手：数据搬运

一张 1080p 图像 = 6 MB

预处理：CPU $\rightarrow$ NPU = 15-20 ms

ViT 输出：3 MB

搬回 CPU：5-10 ms

全程十几次搬运，积累 100+ ms

移动芯片的带宽：

CPU $\leftrightarrow$ DRAM：~50 GB/s（理论）

DRAM $\leftrightarrow$ NPU/GPU：~30-80 GB/s（实际低于理论）

而计算速度：远高于搬运速度

优化思路：减少往返次数

2

五刀优化

每一刀都有收益，也都有代价。

五刀优化方案对比

1. 视觉 Token 裁剪

收益: Prefill -50% | 代价: 小目标精度掉

2. KV Cache 跨帧复用

收益: 重复 prefill -80% | 代价: 场景切换判断复杂

3. W4A8 量化

收益: 内存 -50%, 速度 +30% | 代价: 精度掉 1-3%

4. 异构切分 (ViT→NPU / LLM→GPU)

收益: 两部件并行 +40% | 代价: 数据搬运成本

5. 流水线并行

收益: 吞吐 +2-3x | 代价: 端到端延迟不变

优化 ①：视觉 Token 裁剪

1. 固定分辨率 (336×336 vs 1080×1080)

收益: token 数 -60%, prefill -50%

代价: 细节丢失, 小目标识别下降

2. 动态分辨率 (根据图像内容选择)

收益: 平衡精度和速度

代价: 推理前需分析图像

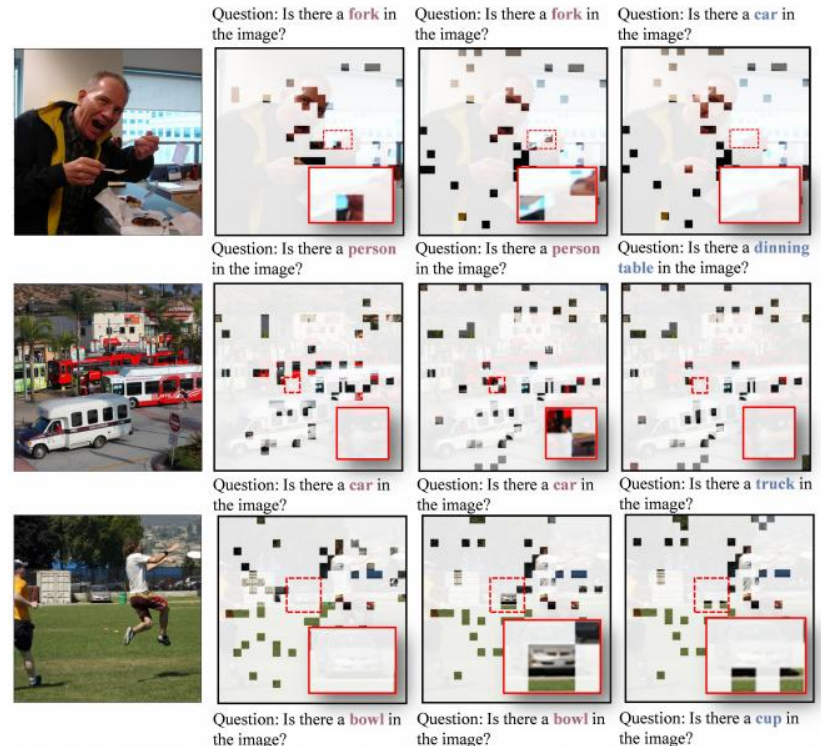
3. ROI 聚焦 (只处理重要区域)

收益: token 数 -70%

代价: 需要目标检测前置

4. Token 稀疏化 (丢弃冗余 patch)

收益: token 数 -40%



Method	Transformation-based	Similarity-based	Attention-based	Query-based
Diagram	 e.g., Pool, Conv	 e.g., KNN unique	 Token Index 1 2 3 4 5 6 1 7 2 3 1 9 2 3 2 4 4 5 3 2 3 8 4 5 4 3 1 1 3 2 2 Mean 7 2 3 3 4 2	 Text Encoder Cross Attention LLM Learned Query Distilled Token Information
Summary	Transform tokens into a more compact form	Compress by merging or grouping similar tokens	Remove less attentive tokens via attention sparsity	Use external queries to guide token compression
Pros	Preserve the structural representation of information well	Simplify processing, flexibility in choosing where to compress	Dynamically prune tokens by relevance; tie to original computation, boosting interpretability	Suitable for specific and video tasks, as compressed information is more relevant and concise
Cons	Limited by the transformation method, compression rate isn't flexible enough	May lose fine-grained info if tokens over-generalized; poor structural feature retention if token differentiation is low	Explicit attention score calculation might be incompatible with mainstream acceleration libraries	Not user-friendly for multi-turn conversations; requires re-condensing information

优化 ②：KV Cache 跨帧复用

背景：机器人是视频流输入 (30 fps)

相邻两帧 95% 像素相同

但 ViT 编码结果每帧都从头计算

优化原理：

相同帧 → 直接复用前一帧的 KV Cache

省掉 prefill 的大头

效果数据：

静态场景：节省 80%

缓变场景：节省 50%

快速变化：无收益

优化 ③：权重 + 激活量化

基础：FP32 $\rightarrow$ INT4 = 8 倍压缩

混精度策略：

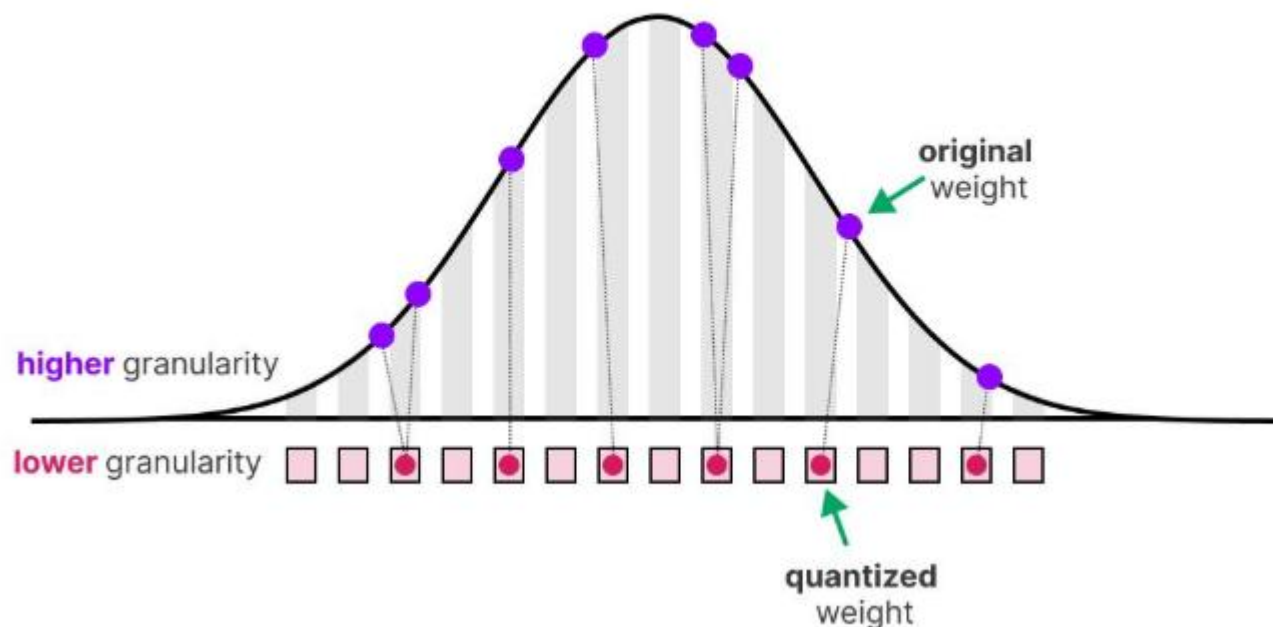
- ViT 权重：INT8（精度敏感）
- LLM 权重：INT4（鲁棒）
- 激活：INT8

真实效果（Qwen-VL 7B on Snapdragon）：

- 内存：12GB $\rightarrow$ 6GB（-50%）
- 速度：800ms $\rightarrow$ 580ms（-28%）
- 精度：评分掉 1-2 个百分点

注意：VLM 对量化敏感，尤其视觉编码部分

大模型量化的核心在于将模型参数的精度从较高的位宽（bit-widths）（例如 32 位浮点数）降低到较低的位宽（bit-widths）（例如 8 位整数）。



优化 ④：异构切分

移动芯片构成：

- CPU：控制、轻量计算
- NPU：矩阵运算（ViT），功耗低
- GPU：并行计算（LLM），功耗高

异构部署：

ViT $\rightarrow$ NPU（200ms）

Prefill $\rightarrow$ GPU（300ms）

Decode $\rightarrow$ NPU（150ms）

理论收益：ViT + Prefill 并行 = 200ms（非 500ms 串行）

实际困难：3MB 数据搬运可能吃掉收益

优化 ⑤：流水线并行

原始流程（串行）：

采集(100ms) + 推理(800ms) + 执行(50ms) = 950ms

流水线化：

Frame_1 采集 + Frame_0 推理 + Frame_-1 执行 并行

效果：

- 吞吐量：1 fps $\rightarrow$ 10 fps（翻倍）
- 端到端延迟：仍然 950ms（无改善）

重要认知：

关心"帧率" $\rightarrow$ 流水线有效 ✓

关心"反应延迟" $\rightarrow$ 流水线无效 ✗

优化陷阱（必读）

陷阱 1：吞吐量 vs 延迟混淆

流水线翻倍吞吐，但延迟无改善 ✕

陷阱 2：单部件 vs 系统优化

ViT 快 40% 但只占 20% → 系统总速度只快 8% ✕

陷阱 3：数据搬运代价被忽视

异构切分 3MB 搬运可能吃掉收益 ✕

陷阱 4：精度掉落的隐性代价

掉 1% 精度 = 小物体识别失效 ✕

陷阱 5：热降频无法绕过

5 分钟内快，但 1 小时后降频反而变差 ✕

3

架构层解法

推理优化有天花板。

7B VLM 不可能以 50Hz

输出精确控制指令。

所以真机系统这样设计：

分层架构：慢系统 + 快系统

高层：VLM (7B)

频率 2-5 Hz | 延迟容忍 200-500ms

任务：理解、规划

中层：策略网络 (50M)

频率 50 Hz | 延迟容忍 20ms

任务：逐步执行规划

底层：PID 控制器

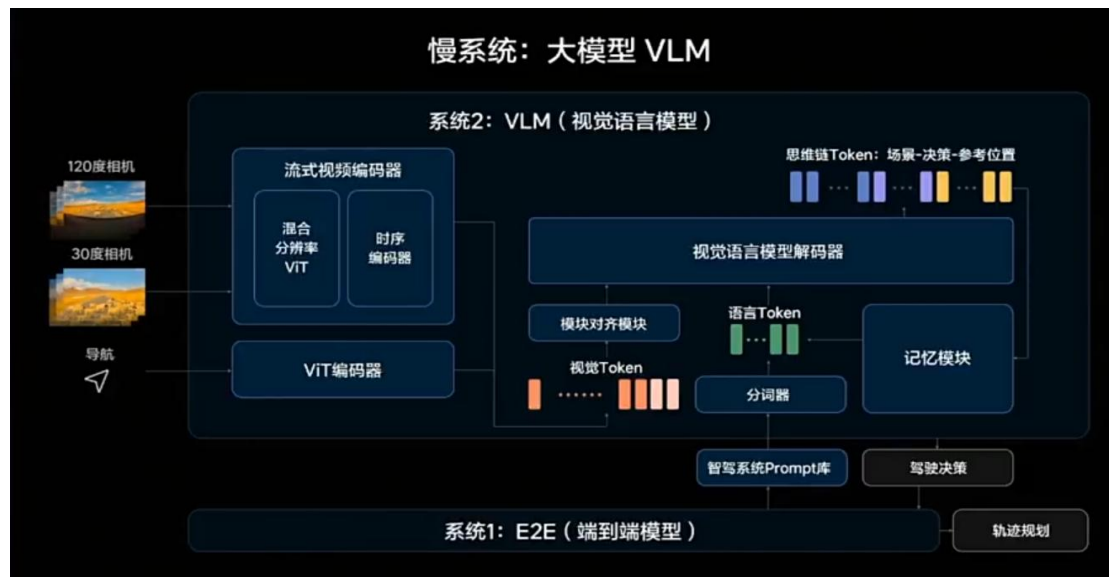
频率 200Hz | 延迟容忍 5ms

任务：物理执行

效果：

✓ VLM 再慢也没关系 (只要 <1s)

✓ VLM 卡顿时 策略网络 hold 住机器人



Action Chunking: 用"预测"换"延迟"

问题: VLM 思考 500ms, 机器人一动不动

解决方案:

VLM 一次预测"未来 10 步" (200ms 序列)

策略网络逐步执行

发现不对时重新问 VLM

伪代码:

```
actions = VLM(image, goal) # 获取 10 步计划
```

```
for i in range(10):
```

```
    motor = policy(actions[i]) # 每 20ms 执行一步
```

```
    if user_interrupt or collision:
```

```
        actions = VLM(image, goal) # 重新规划
```

效果: 用户感受延迟 50ms (策略网络) 而非 500ms (VLM)

端云协同：分界线在哪

云侧场景（网络充足 / 精度第一）：

- 复杂推理、长上下文规划
- 特殊知识推理（查询数据库、API）

端侧场景（立刻返回 / 隐私 / 断网可用）：

- 实时控制反馈（<100ms）
- 动作执行（不能往返网络）
- 异常检测（安全告警）

混合策略（真机常见）：

端侧 VLM：快速响应（有人）

云侧 API：身份识别（谁）

端侧执行：组合结果（动作）

4

上线之后的坑

优化性能只是第一步。

真正的问题在运行时出现。

坑 ①：热降频（功耗墙）

现象：第 1 分钟 800ms，第 10 分钟 1500ms (-50%)

原因：芯片温度 $> 80^{\circ}\text{C}$ $\rightarrow$ 自动降频

对策：

- ① 散热设计（散热孔、导热垫）
- ② 动态功耗管理（关屏幕、限制后台）
- ③ 交替运行（VLM 跑 1 分钟，停 10 秒）
- ④ 软件限频（预先上限设低 20%，维持稳定）

真实测试：Snapdragon 8 Gen 3

前 2 分钟：正常

2-5 分钟：核心温度 75°C ，降频 -10%

5+ 分钟：温度 85°C ，降频 -30%

坑 ②：内存峰值与偶发 OOM

问题分析：

平时推理：6GB（运行时）

峰值：8GB（prefill 中间结果 + KV Cache）

设备总显存：8GB

系统开销：500MB → 实际可用 7.5GB

→ OOM 发生在峰值与系统竞争时

显存碎片化：

[1MB] [空闲 0.5MB] [3MB] [空闲 0.3MB]...

申请 2MB 连续空间却找不到

对策：

① 预分配 KV Cache（启动时一次性申请）

② 减少中间结果拷贝

③ 周期性清理（定时重启推理引擎）

坑 ③：模型异常 → 级联失败

真实案例：机器人跑 3 小时后卡住

[日志] VLM 异常输出："`\\x00\\x01\\xff\\xfe`"

[结果] 解析失败 → 控制命令丢失 → 机器人停止

异常来源：

- 显存损坏 (bits flip)
- 推理引擎内存泄漏导致数据污染
- 特定输入导致 tokenizer 崩溃
- 高温下模型权重数值漂移

对策（兜底机制）：

- ① 输出合法性校验 (token id 范围)
- ② 长度检查 (输出 <Max Length)
- ③ 超时兜底 (>2s 降级默认动作)
- ④ 异常计数 (连续异常 2 次重启引擎)

坑 ④：稳定性的价值

性能优化：大家都关注

「我把速度提升 30%」能量化、能上简历

稳定性：没人关注，但最值钱

「我让系统从 3 小时崩溃变成 7 天不停机」

不好量化，但价值 >> 性能 10 倍

市场现状：

- 性能优化工程师：很多，竞争激烈
- 稳定性工程师：很少
- 大多数系统在"能跑"和"稳定跑"之间选择了前者

稳定性包括：

- 异常检测和恢复
- 资源泄漏的发现和修复

5

项目 + 面试

如何讲清楚你的优化工作

课后项目：做一份完整的优化分析报告

Baseline（原始配置）：

首 token 750ms | 吞吐 20 tokens/s | 内存 7.2GB

拆解（用 profiler 拆时间）：

采集+预处理 150ms | ViT 200ms | Prefill 350ms

优化对比：

1. 降分辨率 336×336

- 首 token 550ms (-26%) | 内存 5.8GB (-19%)
- 代价：小物体精度 F1 92% → 85%

2. W4A8 量化

- 首 token 480ms (-36%) | 内存 3.9GB (-46%)
- 代价：BLIP 分数 8.2 → 7.9

面试官的 5 道追问

Q1: 精度掉 1%, 怎么证明在实际场景有影响

→ 分类别精度对比、A/B 测试、定量结论

Q2: 量化加速 30%, 收益从哪来

→ 访存减少、指令集优化、数据搬运

Q3: 为什么没用知识蒸馏 / 剪枝

→ 成本 vs 收益, 你场景的最优选择

Q4: 热降频怎么处理, 有数据吗

→ 连续运行 30 分钟的性能曲线、采取的措施

Q5: 模型输出异常, 系统如何处理

→ 检测机制 降级策略 恢复 监控

简历怎么写

✗ 不好的写法：

「进行了模型优化工作，使用了量化和分辨率降低技术，提升了系统性能。」

✓ 好的写法：

「在 Snapdragon X Elite 上优化 Qwen-VL 7B 推理：

- 通过 W4A8 量化+降分辨率实现首 token 延迟从 750ms 降至 380ms (-49%)
- 端到端推理时间满足机器人控制环要求 (<100ms)
- 内存占用从 7.2GB 优化至 3.5GB (-51%)
- 在精度掉 1-2 个 F1 点的代价下实现上述收益

技术：ONNX 量化、分辨率自适应、性能 Profiling」

为什么更好：

- ① 具体数字 (750ms → 380ms)
- ② 多个维度 (延迟、吞吐、内存)
- ③ 代价说明 (精度折衷)
- ④ 业务背景 (机器人控制)
- ⑤ 技术细节 (用了什么工具)

总结：完整图景

1. 拆延迟 - 所有优化的起点
2. 多模态特殊性 - 不能照搬 LLM
3. 每一刀都有权衡 - 加速 vs 精度
4. 架构层设计 - 比局部优化重要 10 倍
5. 上线之后才是挑战 - 热、内存、异常
6. 表达能力 - 定量、完整、诚实

做这个方向需要：

- ✓ 深度理解硬件（功耗、热、访存）
- ✓ 细致的测量能力（profiling、基准测试）
- ✓ 系统化思维（从芯片到应用全链路）
- ✓ 工程经验（什么时候追求极限，什么时候停止）

这不只是调参的工作 这是搭建完整系统的工作